Encapsulation

What is Encapsulation?

Encapsulation means bundling **data** (attributes) + **behavior** (methods) into a single unit (a class), and **restricting** direct access to that data.

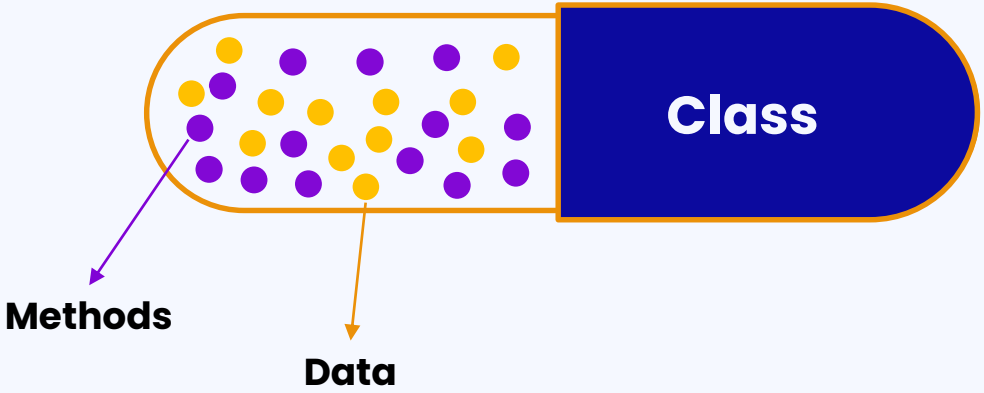
Goal:

- Protect internal state from accidental or unauthorized modification.
- Provide a controlled interface (methods) to interact with the data.

Basically:

You don't touch variables directly. You ask my methods nicely.

Encapsulation Example



“Pythonic“ Encapsulation

Languages like **Java** and **C++** have access modifiers:

- **public**: accessible from anywhere.
- **protected**: accessible within class + subclasses (and sometimes same package).
- **private**: accessible only inside the same class.

Typical pattern:

- **Data (fields)** = private
- **Methods (getters/setters/operations)** = public

Python:

- **No** true access modifiers.
- All **attributes/methods** are **public** by default.
- **Encapsulation** is done via naming conventions and discipline, **not** strict enforcement.

Public

If an attribute/method has **no leading underscore**, it's treated as **public**.

Python

```python
class Book:

    def __init__(self, title, price):
        self.title = title        # public
        self.price = price        # public
```

This **does not** enforce encapsulation, because external code can do:

```python
b = Book("Python Basics", 399)

b.price = -10    # nonsensical, but allowed
```

This is exactly what **strict encapsulation** tries to prevent (which python does not).

Protected (by convention)

- Python uses **single prefix underscores** to make attribute **"protected".**
- For Example: **`_price, _order`** etc.

Python

```python
class Car:
    def __init__(self, model, max_speed):
        self.model = model              # public
        self._max_speed = max_speed     # "protected" by convention

    def show_info(self):
        print(f"{self.model} can go up to {self._max_speed} km/h")

c = Car("Tesla", 250)
c.show_info()           # Allowed & Recommended
print(c._max_speed)     # Technically works, but bad style
```

Private (by convention)

- Python uses **double prefix underscores** to make attribute **"private".**
- For Example: `__price, __order` etc.
- This avoids **accidental access of attribute** that are supposed to be **"internal"**.

Python

```python
class Car:
    def __init__(self, model, max_speed):
        self.model = model              # public
        self.__max_speed = max_speed    # "protected" by convention

    def show_info(self):
        print(f"{self.model} can go up to {self.__max_speed} km/h")

c = Car("Tesla", 250)
c.show_info()           # Allowed & Recommended
print(c.__max_speed)    # AttributeError
```

Name Mangling

- Any **__attribute** becomes **_ClassName__attribute** internally.
- This is not meant as **security**, but a safeguard and convention.
- Can still be accessed **(not recommended):**

Python

```python
class Car:
    def __init__(self, model, max_speed):
        self.model = model              # public
        self.__max_speed = max_speed    # "protected" by convention

    def show_info(self):
        print(f"{self.model} can go up to {self.__max_speed} km/h")

c = Car("Tesla", 250)
c.show_info()           # Allowed & Recommended
print(c.__max_speed)    # AttributeError
c._Car__max_speed       # Works but NOT RECOMMENDED
```

Why Use Encapsulation

- **Protects important data** from being changed directly.
- Lets you **validate** and **control** how data is **updated**.
- **Hides internal details**; users only deal with **simple methods**.
- Allows **internal changes** later **without breaking** other code.

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Inheritance